

Partner4Luv Backend Documentation

Project Overview

This backend is a Python FastAPI application serving a dating/matching platform. It provides:

- REST API routes for user, admin, social, economy, AI, notifications, and chat functionality
- PostgreSQL database access through Prisma ORM
- JWT authentication and password hashing
- Real-time chat through Socket.IO
- AI utilities for profile verification, matching insights, and photo validation
- File upload support for profile and admin images
- Database seeding, validation, and testing scripts

Key Files

``main.py``

- Application entrypoint for FastAPI
- Loads environment variables from ``.env``
- Configures CORS and static file serving for ``uploads/``
- Mounts Socket.IO app on ``/ws``
- Includes routers from ``routes/``
- Manages Prisma DB connect/disconnect on startup/shutdown

``db.py``

- Initializes the Prisma client:
- ``db = Prisma()``

``auth_utils.py``

- Handles JWT token creation and secret management
- Uses ``passlib`` for password hashing and verification
- Uses ``python-jose`` for signing JWT tokens

``ai_utils.py``

- Wraps OpenAI API calls for profile auditing, match insights, and photo verification
- Contains ``AIClient`` with methods:
- ``verify_profile``
- ``get_matching_insight``
- ``verify_photo``

``ai_insights.py``

- Provides helper functions for AI-powered matching scores and insights

``socket_manager.py``

- Implements Socket.IO server events:
- ``connect``
- ``disconnect``
- ``join_chat``

- ``send_message``
- ``typing_status``
- Persists chat messages and conversation updates in the database

``stripe_utils.py``

- Stripe integration helper utilities for payments and subscriptions

``subscription_utils.py``

- Subscription plan helpers and business logic

Folders

``prisma/``

- ``schema.prisma`` defines the PostgreSQL data model
- Models include:
 - ``User``, ``Profile``, ``Photo``
 - ``Match``, ``Conversation``, ``Message``, ``Block``, ``Report``
 - ``QuestionnaireCategory``, ``Question``, ``QuestionOption``, ``UserResponse``
 - ``Payment``, ``SubscriptionPlan``, ``Notification``, ``Faq``, ``AuditLog``, ``AdminProfile``

``routes/``

- Contains grouped FastAPI routers
- ``admin.py``: admin login, profile, FAQ, user management, password reset
- ``ai.py``: AI endpoints for profile verification and matchmaking
- ``chat.py``: chat and conversation APIs
- ``economy.py``: payments, plans, and wallet/economy endpoints
- ``notifications.py``: notifications CRUD and delivery
- ``social.py``: social actions, matching, swipes, blocks
- ``user.py``: user auth, signup, profile setup, social login, password reset

``scripts/``

- Utility and maintenance scripts:
 - ``check_links.py``, ``check_questions.py``, ``check_routes.py``
 - ``debug_user.py``, ``fix_links.py``
- Seeders: ``seed.py``, ``seed_logs.py``, ``seed_more_users.py``, ``seed_plans.py``, ``seed_questionnaire.py``, ``seed_questions.py``, ``seed_test_users.py``, ``seed_users.py``, ``seed_user_ansh.py``
- Tests: ``test_api.py``, ``test_prisma_models.py``

``uploads/``

- Static file storage for uploaded content
- Mounted by ``main.py`` at ``/uploads``

``utils/``

- ``ai_tagger.py``: AI tag generation and categorization utilities

Dependencies

From ``requirements.txt``:

- `fastapi`, `uvicorn`, `prisma`, `python-dotenv`
- `python-jose`, `passlib`, `bcrypt`
- `python-socketio`, `websockets`, `python-multipart`
- `openai`, `stripe`
- `pydantic`, `starlette`, `httpx`

Running the Backend

1. Ensure environment variables are defined in `.env`
2. Start the server with `python main.py` or `uvicorn main:app --reload`
3. Prisma will connect to PostgreSQL on startup
4. Static uploads are available under `/uploads/`

Notes

- The backend uses Prisma Client for Python with the schema defined in `prisma/schema.prisma`
- Real-time chat is implemented through Socket.IO and saved to the database
- Admin routes are under `/auth/admin` while user routes use `/auth/user` and `/api`
- AI utilities rely on OpenAI keys from environment variables